

function Approximation —

Parameterized functions :-

- # So far, our value approximations have always had the same form. For each state, we store separate value in a table. We look at values, and modify them in the table as learning progresses.
- # But this is not the only way to approximate a value function. In principle, we can use any function that takes a state, and produces a real number.
- # represented by $\hat{v}(s, w)$, where w is a vector containing all the weights that ~~approximate the true value function~~ parameterized the approximation.

Linear Value function approximation →

- ∅ In this case, the value of each state is represented by a linear function of the weights.

$$\hat{v}(s, w) = \sum w_i x_i(s)$$

- ∅ This simply means that ~~that~~ the value of each state, is computed as the sum of weights multiplied by some fixed attributes of the states called features!

$$\hat{v}(s, w) \doteq \sum w_i x_i(s)$$

$$= \langle w, x(s) \rangle$$

↑
feature vector

linear function approximation is very general. Even a tabular representation is a special case of linear function approximation.

To build a linear value function to represent this tabular value function, we need to define the features.

let's choose our features to be indicator functions for particular states. for state s_i , feature i is one and remaining features are 0.

$$x(s_i) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \leftarrow \text{ith element}$$

hence

$$\hat{v}(s_i, w) = \langle w, x(s_i) \rangle$$

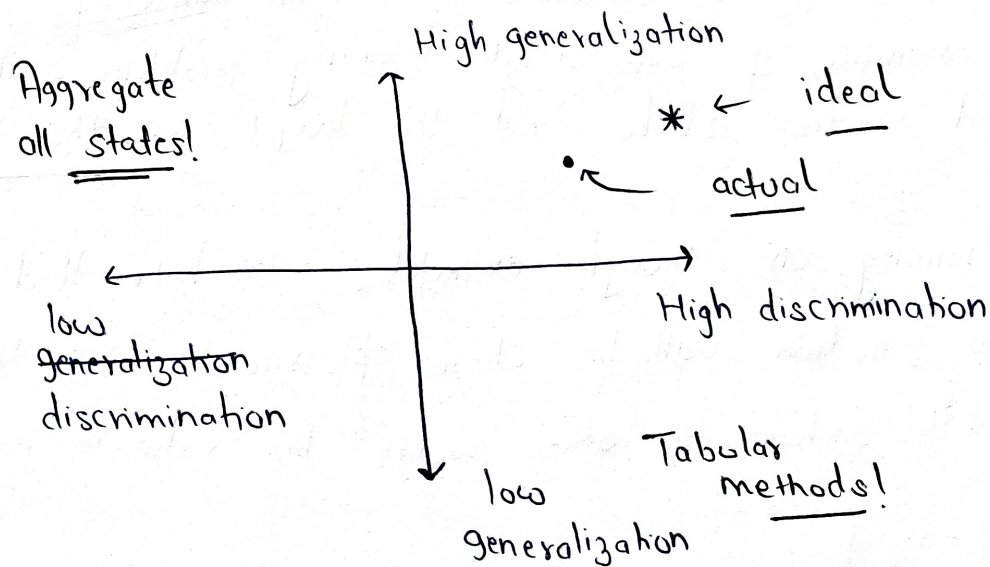
$$\boxed{\hat{v}(s_i, w) = w_i}$$

Neural Network are example of a non-linear function of state.

- # The output of the network is our approximation approximate value for a given state. ($\hat{v}(s, w)$)
- # The state is passed to the network as the input.

Generalisation and discrimination →

- # Generalization in context of policy evaluation, we mean that ~~updates~~ ~~the~~ updates to the value ~~estimates~~ of one state influence the value of other states. Thus, we want the value function to generalize across those states. Generalization can speed learning by making better use of the experience we have.
- # Discrimination means the ability to make values for two states different to distinguish b/w the values for these two states.



- # Tabular methods discriminate b/w states perfectly, but do not generalize the learn values at all, each value is represented independently.

aggregate all states :- Here we treat all states as same. each update generalises to all states. At best, we will be able to learn the average return independent of the current state.

Framing Value Estimation as Supervised learning →

- ✓ Supervised learning involves approximating a function given a dataset of input target pairs.
- ✗ The problem of policy evaluation in RL can be framed similarly. This similarity is most obvious in case of Monte Carlo methods.
- ✗ MC methods estimate the value functions using samples of the returns. We can think of this as an example of supervised learning problem, where the input is the state and the targets are the returns.
- ✗ By training on enough examples, we hope that our learn function will be close approximation of the expected return. In other words, the value in each state.

$\{(s_1, r_1),$

$(s_2, r_2),$

$(s_3, r_3),$

$\dots\}$

Monte-Carlo

∅ TD Can also be framed as supervised learning, where targets are the one-step bootstrap return. (40)

$$\left\{ \begin{aligned} &(s_1, R_1 + \gamma V(s_2, w)), \\ &(s_2, R_2 + \gamma V(s_3, w)), \\ &\dots \end{aligned} \right\} \quad \underline{\underline{TD}}$$

∅ Any function approximation technique from supervised learning can be applied to the policy evaluation task, however, not all are equally well suited.

∅ In RL, agent interacts with the environment and continually generates new data. This is often called the online setting.

∅ To distinguish from it from offline setting where the full dataset is available from the starting and remain fixed throughout the learning.

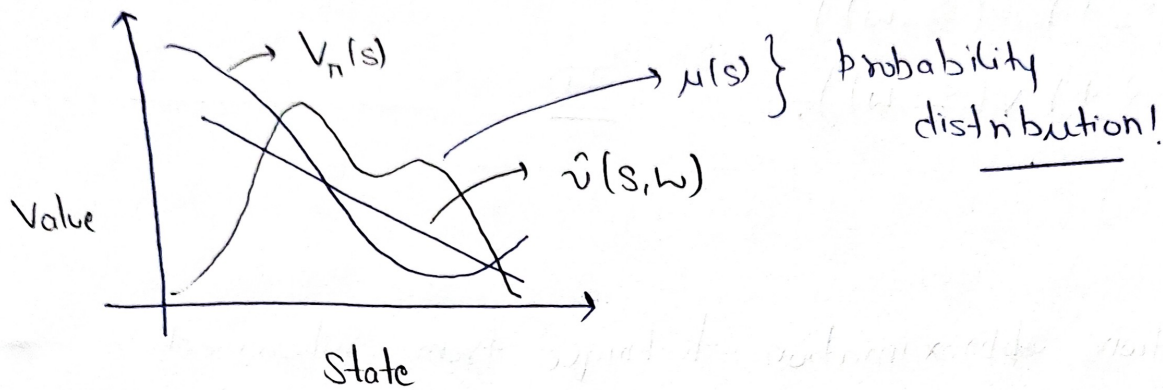
∅ Therefore, our function approximation technique should work with online setting!

∅ TD methods introduce an additional complication when applying techniques from supervised learning.

∅ TD methods use bootstrapping, meaning that our targets now depend on our own estimates. These estimates changes as learning progresses and so

our targets continually change.

Mean Squared Value Error Objective →

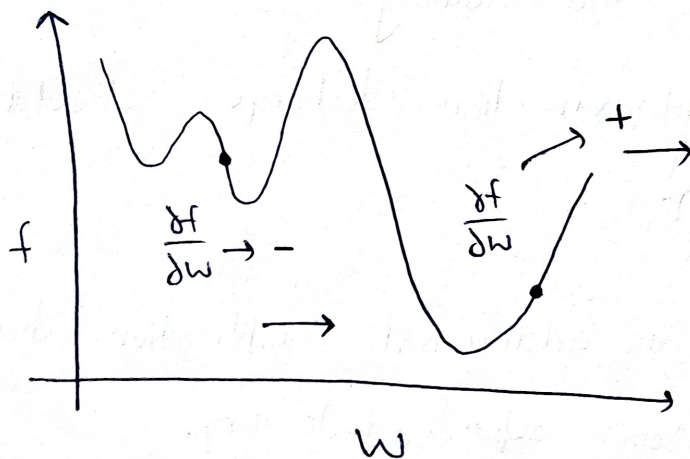


$$\bar{VE} = \sum_s \underbrace{\mu(s)}_{\text{how much we care about each state!}} [V_{\pi}(s) - \hat{v}(s, w)]^2$$

how much we
care about each state!

The fraction of time
we spend in state
 s when following
policy π .

Gradient Descent →

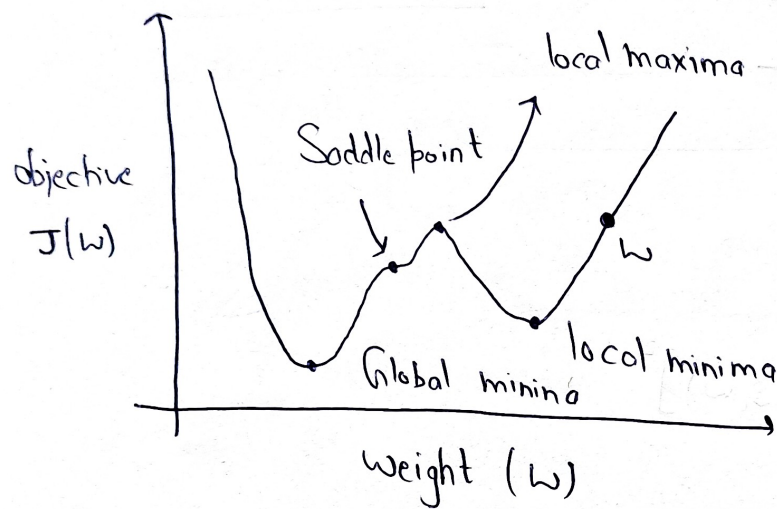


Gradient := Derivatives in multiple dimensions!

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \dots \\ w_d \end{bmatrix} \quad \nabla f = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \dots \\ \frac{\partial f}{\partial w_d} \end{bmatrix}$$

- ⊘ The direction to change w_2 in order to increase f
- ⊘ How quickly f changes

⊘ The gradient gives the direction of steepest ascent!



$$w_{t+1} = w_t - \alpha \nabla J(w_t)$$

$\uparrow$
Step size parameter

why this
-ve sign here!

⊘ In some cases, the gradient descent is ~~not~~ guaranteed to converge to the global minima, which is best possible setting of the weights for the objective.

⊘ This is true for the mean squared value error with linear ~~value~~ approximation.
function

∅ Global minima does not necessarily corresponds to the true value function. It is limited by our choice of function parameterization, and depends on our choice of objective.

$$\hat{v} \neq \underline{v_\pi}$$

∅ Gradient of the Mean Squared Value Error Objective —

$$\nabla \sum_{s \in \mathcal{S}} \mu(s) [v_\pi(s) - \hat{v}(s, \omega)]^2$$

$$= \sum_{s \in \mathcal{S}} \mu(s) \nabla [v_\pi(s) - \hat{v}(s, \omega)]^2$$

$$= - \sum_{s \in \mathcal{S}} \mu(s) 2 [v_\pi(s) - \hat{v}(s, \omega)] \nabla \hat{v}(s, \omega)$$

⇒ In Case, $\hat{v}(s, \omega) = \langle \omega, x(s) \rangle$
 $\nabla \hat{v}(s, \omega) = x(s)$ } (linear function approximation

$$= - \sum_{s \in \mathcal{S}} 2 \mu(s) x(s) [v_\pi(s) - \hat{v}(s, \omega)]$$

∅ The gradient of the value function approximation will indicates how to change the weights to increase the value for that state.

- ∅ We multiply this direction by the difference b/w the true value and our estimate.
- ∅ If the difference is +ve, it means true value is higher than our estimate, so we should change ~~our estimate~~ the weights in the direction that increases our estimate.

∅ Computing

$$\sum_{s \in \mathcal{S}} \underbrace{\mu(s)} \alpha [V_{\pi}(s) - \hat{v}(s, w)] \nabla \hat{v}(s, w)$$

is not feasible generally — generally we don't know this!

∅ Therefore, we will approximate this!

- ∅ let we have access to V_{π} in an idealized setting. Though we do not have $\mu(s)$, we can sample states from it simply by following the policy.

$$(s_1, V_{\pi}(s_1)), (s_2, V_{\pi}(s_2)), (s_3, V_{\pi}(s_3)) \dots$$

$$\left. \begin{aligned} w_2 &\doteq w_1 + \alpha [V_{\pi}(s_1) - \hat{v}(s_1, w_1)] \nabla \hat{v}(s_1, w_1) \\ w_3 &\doteq w_2 + \alpha [V_{\pi}(s_2) - \hat{v}(s_2, w_2)] \nabla \hat{v}(s_2, w_2) \end{aligned} \right\}$$

∅ above approach is called stochastic gradient descent,

because it only uses a stochastic estimate of the gradient.
the expectation of each stochastic gradient equals the
gradient of the objective.

- ∅ Stochastic gradient can be think as noisy approximation of the gradient
- ∅ Stochastic gradient descent allowed us to efficiently update the weights on each step by sampling the gradient.
- ∅ However there one practical issue here, we do not have access to $v_{\pi}(s)$!
- ∅ let's replace v_{π} with an estimate. One option is to use samples of the return for each state visited.

$$w_{t+1} \doteq w_t + \alpha \left[\underbrace{v_{\pi}(s_t)} - \hat{v}(s_t, w_t) \right] \nabla \hat{v}(s_t, w_t)$$

$$w_{t+1} \doteq w_t + \alpha \left[\underbrace{G_t} - \hat{v}(s_t, w_t) \right] \nabla \hat{v}(s_t, w_t)$$

$$\therefore \text{Since } v_{\pi}(s) \doteq \mathbb{E}_{\pi} [G_t | s_t = s]$$

what is unbiased estimator?

In fact,

$$\begin{aligned} & \mathbb{E}_\pi \left[2 \left[G_t - v(s_t, \omega) \right] \nabla v(s_t, \omega) \right] \\ &= \mathbb{E}_\pi \left[2 \left[v_\pi(s_t) - \hat{v}(s_t, \omega) \right] \nabla \hat{v}(s_t, \omega) \right] \end{aligned}$$

Gradient Monte Carlo Algorithm for estimating $\hat{v} = v_\pi \rightarrow$

- 1) Input : the policy π to be evaluate
- 2) Input : a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$
- 3) Algorithm parameter : step size, $\alpha > 0$
- 4) Initialize : Value function weights $\omega \in \mathbb{R}^d$ arbitrarily.
- 5) Loop forever (for each episode):
 - 6) Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π .
 - 7) Loop for each step of the episode,
 $t = 0, 1, \dots, T-1$
 - 8) ~~Update~~ $\omega \leftarrow \omega + \alpha \left[G_t - \hat{v}(s_t, \omega) \right] \nabla \hat{v}(s_t, \omega)$